Voice MFA System

Multi-Factor Authentication with Voice Biometrics

FastAPI | React | Resemblyzer | Redis | JWT | TOTP | Docker

Computer Networks and Security Project | May 2026

Project Overview

What It Is

Layered authentication system combining:

- Password (knowledge factor)
- Voice biometrics (biometric factor)
- TOTP (possession factor)

Two implementations:

- Local Python prototype (CLI + Tkinter)
- Full-stack web application

Tech Stack

- Frontend: React + Vite
- Backend: FastAPI (Python)
- Voice AI: Resemblyzer (256-dim embeddings)
- OTP: pyotp (TOTP standard)
- Auth: JWT (pre-auth + full-session)
- Storage: SQLAlchemy (SQLite / PostgreSQL)
- Cache: Redis (challenge phrases)
- Containers: Docker Compose

Problem Statement & Motivation

81%

of breaches involve compromised passwords

3x

more secure with multi-factor auth

99.9%

of account attacks blocked by MFA

Core Problems with Password-Only Auth

Phishing

Users tricked into revealing credentials on fake sites

Credential Reuse

Same password reused across multiple services

Data Breaches

Hashed passwords exposed from compromised databases

Brute Force

Automated guessing of weak or short passwords

MFA Factors: The Three Pillars

KNOWLEDGE

Something You Know

Username & Password

Hashed with bcrypt before storage. Never stored as plain text.

BIOMETRIC

Something You Are

Voice Biometrics

256-dimensional Resemblyzer embedding. Cosine similarity verification.

POSSESSION

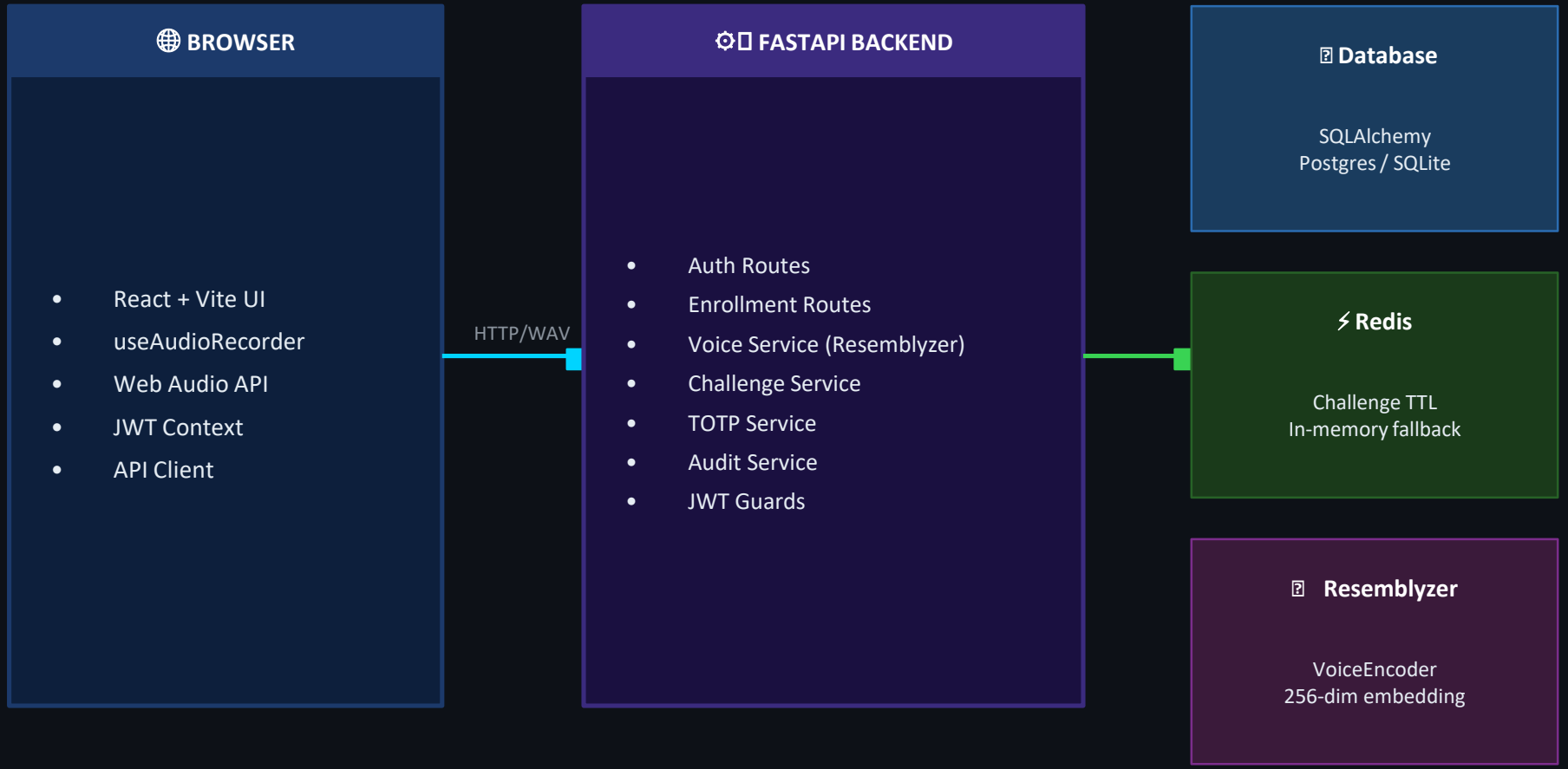
Something You Have

TOTP (Authenticator App)

Time-based one-time password. Verified via pyotp. 6-digit code.

✦ *All required factors must pass before a full session is granted* ✦

System Architecture



Technologies Used

Frontend

- React 18 + Vite
- Web Audio API
- qrcode.react
- CSS Modules

Backend

- FastAPI (Python)
- Pydantic Settings
- SQLAlchemy ORM
- Uvicorn ASGI

Biometrics

- Resemblyzer
- NumPy (embeddings)
- SoundFile (WAV I/O)
- Cosine Similarity

Security

- bcrypt (hashing)
- python-jose (JWT)
- pyotp (TOTP)
- Redis (challenges)

Storage

- SQLite (dev)
- PostgreSQL (prod)
- Redis (cache)
- Pickle (local)

DevOps

- Docker Compose
- Python 3.11-slim
- Node 20 Alpine
- LibSndFile + FFmpeg

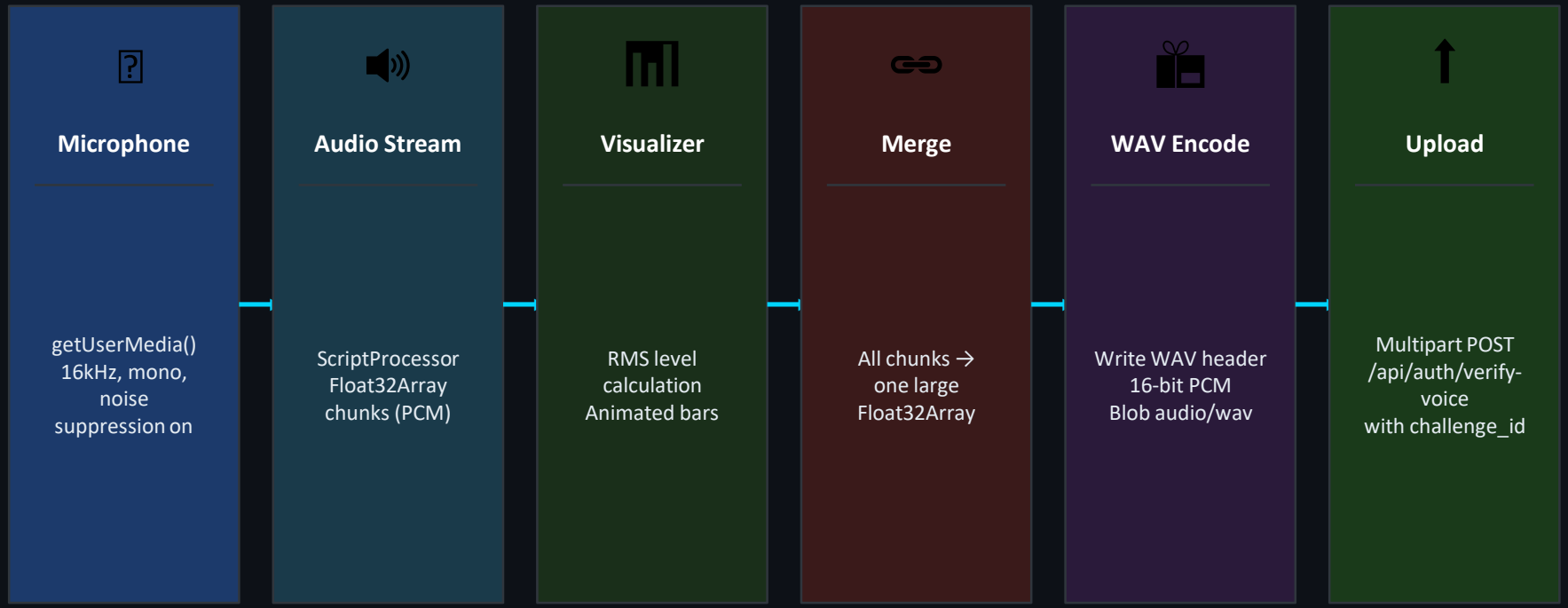
Local Proto

- sounddevice (audio)
- Vosk STT (offline)
- Tkinter (GUI)
- RPi GPIO

Standards

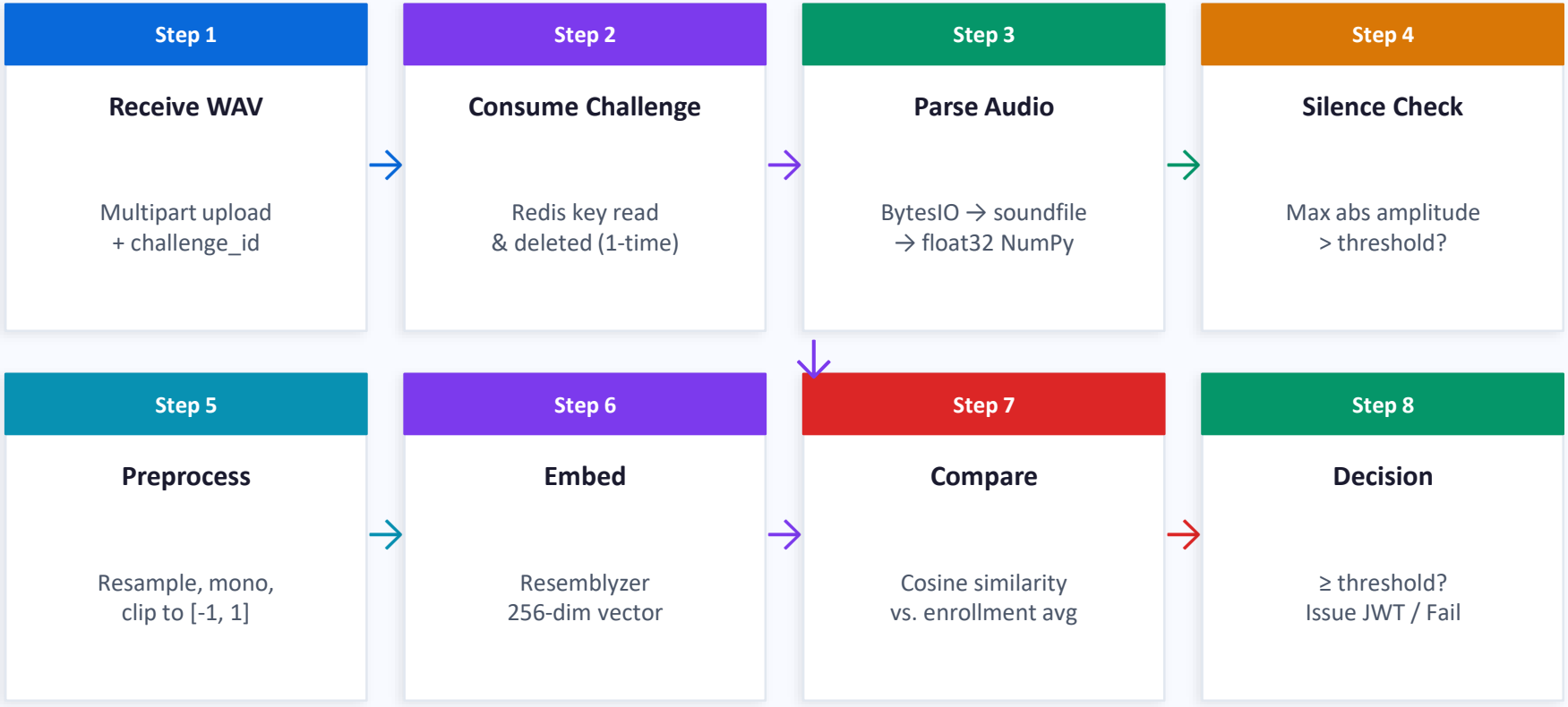
- RFC 6238 (TOTP)
- 16kHz mono WAV
- Bearer JWT
- CORS + HTTPS

Frontend: Audio Recording Pipeline



📌 *The frontend has one responsibility: capture clean audio and deliver a WAV Blob to the backend. It does NOT perform voice verification — that logic lives exclusively in the backend.*

Backend: Voice Verification Pipeline



Voice Enrollment Process

1

Record Samples

Min. 3 samples (web) or 5 (local). Varied phrases encouraged for a richer voice profile.

2

Signal Cleaning

Convert to mono → remove NaN/Inf → clip to [-1, 1] → boost quiet audio.

3

Generate Embeddings

Resemblyzer preprocess_wav → VoiceEncoder.embed_utterance() → 256-dim vector per sample.

4

Store in Database

Each embedding serialized via np.save and stored in voice_profiles table (web app).

5

Complete Enrollment

is_voice_enrolled = True. Future logins will require voice verification.

Why Multiple Samples?

- One sample may be noisy or unrepresentative
- Multiple samples create a stable voice profile
- Averaged embeddings are more robust
- Reduces false rejections for legitimate users

- Web app: each embedding stored separately, averaged at verification time

- Local prototype: mean embedding saved to enrolled_users/<user>.pkl immediately

Voice Verification Algorithm

ENROLLMENT PHASE

1. Record N audio samples
2. Convert to float32 NumPy array
3. Clean signal (mono, clip, normalize)
4. preprocess_wav (Resemblyzer)
5. embed_utterance → 256-dim vector
6. Store each embedding in DB

VERIFICATION PHASE

1. Get fresh challenge from Redis
2. Record new voice sample (browser)
3. Consume challenge ID (prevent replay)
4. Extract new 256-dim embedding
5. Load & average enrollment embeddings
6. Compute cosine similarity → compare vs. threshold

Voice Embeddings & Cosine Similarity

What is a Voice Embedding?

A compact mathematical fingerprint of a speaker's voice

- 256 floating-point numbers per sample
- Numbers are not individually meaningful
- The entire vector encodes speaker identity
- Same speaker → similar directions in vector space

Why not compare raw audio?

Two recordings from the same person NEVER have the same waveform. Volume, speed, mic, noise — all affect the signal. Embeddings are robust to these variations.

Cosine Similarity

$$\text{similarity} = (A \cdot B) / (||A|| \times ||B||)$$

- Measures the angle between two vectors
- Score range: 0.0 (orthogonal) → 1.0 (identical)
- Robust to magnitude differences (volume)

Score ≥ 0.80 ✓ Accept (same speaker)

Score < 0.80 ✗ Reject (different speaker)

Threshold is a security/usability tradeoff:

- Higher threshold → stricter but more false rejects
- Lower threshold → more convenient but risk of false accepts

Challenge Phrase & Replay Attack Mitigation

How It Works

1. **Generate**
   Backend creates: 4 random words + 3-digit number
   Example: "glacier raven ember 742"
2. **Store in Redis**
   challenge:<uuid> key with short TTL
   (CHALLENGE_EXPIRE_SECONDS)
3. **Display to User**
   Frontend shows phrase + countdown timer
   User reads it aloud
4. **Upload + Consume**
   Audio + challenge_id sent to backend
   Redis key is read AND deleted immediately
5. **One-Time Use**
   Expired or reused challenge_id → immediate reject
   Prevents recording replay attacks

⚠ Limitation

- Web backend does NOT transcribe spoken audio
- The challenge acts as an anti-replay token — not as liveness proof

Local Prototype has Vosk STT:

- Transcribes audio and fuzzy-matches spoken words vs expected phrase

Improvement needed:

- Add Vosk/Whisper to web backend for full spoken-content verification

Two-Stage JWT Authentication

JWT Payload

```
sub: "user_id"
username: "alice"
type: "pre_auth"
        OR
type: "full_session"
iat: <issued_at>
exp: <expiry>
```

Key Security Principle: "Password accepted" ≠ "User fully authenticated." The pre-auth token allows only MFA-related API calls. Full access is granted only when ALL required factors pass — this design is essential for proper MFA systems.

TOTP & Backup Codes

TOTP Setup & Verification

1. Backend generates a random base32 secret
2. Builds otpauth:// provisioning URI
3. Frontend renders URI as QR code (qrcode.react)
4. User scans with authenticator app (Google Auth, Authy)
5. User enters current 6-digit code to confirm setup
6. Backend verifies with pyotp (allows small time window)
7. TOTP enabled → backup codes generated & hashed

Backup Codes (Recovery)

Why backup codes?

- User loses authenticator app → recovery path needed

Implementation:

- Generated with secrets.token_hex()
- Shown to user EXACTLY ONCE (raw codes)
- Database stores only bcrypt hashes
- Each code is single-use (marked as used)

Security properties:

- Hashed → safe even if DB is breached
- Single-use → prevents code reuse

Database Schema & Data Structures

users

- id (GUID PK)
- username (unique)
- email (unique)
- password_hash (bcrypt)
- is_voice_enrolled (bool)
- is_totp_enabled (bool)
- totp_secret
- failed_attempts
- locked_until
- created_at

voice_profiles

- id (GUID PK)
- user_id (FK → users)
- embedding (BYTES)
- sample_number
- created_at

→ np.save() serializes the 256-dim vector

backup_codes

- id (GUID PK)
- user_id (FK → users)
- code_hash (bcrypt)
- is_used (bool)
- used_at
- created_at

→ raw codes never stored

audit_logs

- id (GUID PK)
- user_id (nullable FK)
- event_type (string)
- ip_address
- user_agent
- details (JSON)
- created_at

→ REGISTER, LOGIN, LOCKOUT, VOICE_VERIFY_SUCCESS/FAIL

Tokens stored in browser localStorage (full-session JWT). Redis stores challenge phrases with TTL — not in the relational DB.

Security Features & Strengths

bcrypt Password Hashing

Salted, slow hashing resistant to brute-force and rainbow table attacks. Never stored in plaintext.

Voice Embeddings

Resemblyzer extracts 256-dim speaker fingerprints — robust to recording variations.

Cosine Similarity

Vector comparison ignores magnitude — handles volume and mic differences naturally.

One-Time Challenges

Redis-backed challenge IDs consumed on use. Expired or replayed IDs are immediately rejected.

Two-Stage JWT

Pre-auth token grants only MFA access. Full token issued only after all factors pass.

TOTP (RFC 6238)

Time-based OTP from authenticator app. Works offline. Verified with pyotp with clock tolerance.

Account Lockout

Failed voice attempts tracked. Exceeded threshold → account locked for configurable duration.

Audit Logging

All security events recorded: registration, login, enrollment, verification, lockout.

Security Limitations & Honest Assessment

MEDIUM	**No Spoken-Content Verification (Web)** The web backend verifies speaker identity but does NOT transcribe the spoken challenge. An attacker could submit any fresh recording of the user's voice and pass.
HIGH	**No Anti-Spoofing / Liveness Detection** Resemblyzer is a speaker similarity model — not an anti-spoofing model. High-quality voice cloning, deepfake audio, or a recording played through speakers could potentially fool it.
LOW	**JWT in localStorage** Full-session token stored in browser localStorage is exposed to XSS if the frontend has vulnerabilities. Hardened systems use httpOnly cookies.
LOW	**No Email Verification** New accounts are activated immediately without confirming email ownership.
LOW	**No Global Rate Limiting** Account lockout exists for voice failures, but login and challenge endpoints lack IP-based rate limiting.
INFO	**In-Memory Challenge Fallback** Redis fallback is process-local — unsuitable for multi-worker deployments. Real Redis required for production.

Possible Improvements & Future Work

P: HIGH

Server-Side Speech-to-Text

Integrate Vosk or OpenAI Whisper in the web backend to verify that the user actually spoke the challenge phrase. Local prototype code already provides a blueprint.

P: HIGH

Anti-Spoofing / Liveness Detection

Add a model that distinguishes live speech from recordings, deepfakes, or text-to-speech synthesis.

P: MED

IP-Based Rate Limiting

Add rate limits on login, challenge, and verification endpoints to prevent automated attacks beyond account lockout.

P: MED

Secure Cookie Storage

Replace localStorage JWT storage with httpOnly, Secure cookies to eliminate XSS exposure.

P: MED

Session Management UI

Let users view active sessions, device information, and revoke sessions remotely.

P: LOW

Email Verification

Confirm email ownership during registration before activating the account.

P: LOW

HTTPS Enforcement

Enforce HTTPS in production. Required for microphone access in real deployments.

P: LOW

Threshold Calibration

Collect FAR/FRR data across microphones, accents, and environments to choose an evidence-based similarity threshold.

Local Prototype vs. Web Application

Feature	Local Prototype	Web Application
Interface	CLI menu + Tkinter GUI	React browser UI
Audio Input	sounddevice (local mic)	Web Audio API (browser)
Voice Model	Resemblyzer (same)	Resemblyzer (same)
Storage	Pickle files (.pkl)	SQLAlchemy DB
Challenge Check	Vosk STT — verifies spoken words	One-time UUID — no STT
TOTP Support	Not implemented	Full pyotp integration
Account Mgmt	Flat user profiles	Users, Lockout, Audit Logs
Deployment	python main.py	Docker Compose
Hardware	PC + optional RPi GPIO	Any browser + cloud server
Use Case	Smart door / embedded demo	Real account MFA flow

Both implementations share the same core idea: Resemblyzer embeddings + cosine similarity. The web version extends it into a full production-like

Deployment with Docker Compose

postgres

Port: 5432

Relational DB
Users, embeddings,
backup codes, audit logs

redis

Port: 6379

Challenge cache
Key: challenge:<uuid>
TTL per challenge

backend

Port: 8000

FastAPI + Uvicorn
Python 3.11-slim
libsndfile + ffmpeg

frontend

Port: 3000

React + Vite
Node 20 Alpine
/api → proxy to :8000

Key Environment Variables (.env)

```
DATABASE_URL=postgresql://...
REDIS_URL=redis://redis:6379/0
JWT_SECRET_KEY=<secret>
SIMILARITY_THRESHOLD=0.72
CHALLENGE_EXPIRE_SECONDS=120
LOCKOUT_DURATION_MINUTES=30
```

SQLite + in-memory Redis available for local development without Docker. Docker Compose adds PostgreSQL and real Redis for production-like testing.

End-to-End: Alice's Authentication Flow

REGISTER

alice / alice@example.com / password123 → bcrypt hash stored

ENROLL VOICE

3+ WAV samples → embeddings stored in voice_profiles table → is_voice_enrolled = True

GET CHALLENGE

GET /api/challenge → {id: "9b8a...", phrase: "glacier raven ember 742", ttl: 120}

FAIL CASE

cosine(0.61) < 0.80 → VOICE_VERIFY_FAILED → failed_attempts++ → lockout if limit reached

FIRST LOGIN

No MFA yet → Full-session token issued immediately

LOGIN AGAIN

Password OK → Pre-auth token + requires_voice=true

VOICE VERIFY

Record 6s → WAV → embed → cosine(0.84) ≥ 0.80 → PASS → Full-session token

DASHBOARD

Full-session JWT → Protected endpoints accessible → Audit log visible

Key Results & Main Achievements

2

Implementations

Local CLI/GUI + Full-stack web app

256

Dimensions

Per voice embedding (Resemblyzer)

3

Auth Factors

Password + Voice + TOTP

4

DB Tables

users, voice_profiles, backup_codes, audit_logs

Technical Milestones

- ✓ Browser-to-backend voice pipeline: Web Audio API → WAV Blob → FastAPI → NumPy → Resemblyzer
- ✓ Voice embedding enrollment and verification with configurable cosine similarity threshold
- ✓ Two-stage JWT system separating password acceptance from full MFA authentication
- ✓ Redis-backed one-time challenge system preventing audio replay attacks
- ✓ TOTP setup with QR code, pyotp verification, and bcrypt-hashed single-use backup codes
- ✓ Account lockout on excessive voice verification failures with configurable thresholds
- ✓ Full audit logging with structured event types, IP recording, and user-visible dashboard
- ✓ Docker Compose deployment with PostgreSQL, Redis, FastAPI, and React in four services

Conclusion

"The user logs in, speaks, the system converts the voice into a mathematical fingerprint, compares it with enrolled samples, and grants access only when all required checks pass."

Technical Depth

Full-stack architecture connecting browser audio, Python AI, relational DB, Redis, and JWT authentication

Security Thinking

Every decision — bcrypt, embeddings, challenges, two-stage tokens — reflects genuine security reasoning

Path Forward

Vosk STT + anti-spoofing detection are the clear next steps toward production-grade voice MFA

Thank You | Questions Welcome

References & Technologies

Voice AI

- Resemblyzer — Speaker verification via d-vector embeddings (Corentin Jemine)
- VoiceEncoder.embed_utterance() — 256-dim speaker embedding
- Vosk — Offline speech recognition (local prototype only)

Frontend

- React 18 + Vite — UI framework and build tool
- Web Audio API — Browser audio capture spec (W3C)
- qrcode.react — QR code rendering for TOTP provisioning URIs

Security Standards

- RFC 6238 — TOTP: Time-Based One-Time Password Algorithm
- bcrypt — Adaptive password hashing (Niels Provos & David Mazières)
- JWT (JSON Web Tokens) — RFC 7519
- OWASP MFA Cheat Sheet

Infrastructure

- Docker Compose — Multi-container orchestration
- PostgreSQL — Production relational database
- Redis — In-memory data store for challenge TTLs
- Uvicorn — ASGI server for FastAPI

Frameworks & Libraries

- FastAPI (Sebastián Ramírez) — async Python web framework
- SQLAlchemy — Python ORM
- pyotp — Python TOTP/HOTP implementation
- python-jose — JWT library for Python